DAY 11 | HANDS-ON ACTIVITIES

React UI Logic and Full Stack: Problem Statements

Seven problems building the SmartDesk full stack. In Colab, use show_react() and the served app from the notebook.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Login Banner

Level: Warm-up | **Concepts:** ternary, &&

Build a component with a loggedIn state and a toggle button. Show "Welcome back, Mei" or "Please log in", and show a badge with the open ticket count only when it is above 0.

Sample run / expected result

```
Welcome back, Mei [3 open]
... toggle ...
Please log in
```

Hints

- {open > 0 && ...}

Problem 2: Ticket List

Level: Easy | **Concepts:** map, keys

Render an array of 6 tickets as cards, sorted by priority (high first). Closed tickets show "Closed" and a greyed style.

Sample run / expected result

```
T-101 No signal at home (high)
T-104 Slow 5G data (medium)
T-103 Change plan (Closed)
```

Hints

- Sort a copy: [...tickets].sort(...)

Problem 3: Search and Filter

Level: Medium | **Concepts:** controlled input, derived list

Add a search box and a "Hide closed" checkbox to the ticket list. Show "No tickets match" when the filtered list is empty.

Sample run / expected result

```
Search: "sim" -> SIM not activated
No tickets match
```

Hints

- Derive visible tickets during render; no extra state.

Problem 4: Four States

Level: Medium | **Concepts:** useEffect, loading, error, empty

Create a fake API function that resolves after 1 second, randomly fails 30 percent of the time, and sometimes returns an empty list. Show a loading, error (with Retry), empty and data view.

Sample run / expected result

```
Loading tickets...
Could not load tickets [Retry]
No tickets yet
3 tickets loaded
```

Hints

- Retry can increment a counter used in the useEffect dependency array.

Problem 5: Find the Leaked Key NEW (v4)

Level: Medium | **Concepts:** build output, secret scanning

Use the notebook's simulated Vite build: put a fake key in a VITE_ variable, build, and write scan_for_secrets(folder) that reports every file and line that looks like a key. Then move the call behind a FastAPI endpoint and show that the new bundle is clean.

Sample run / expected result

```
Before: dist/assets/app.js:3 looks like an API key
After: no secrets found in dist/
```

Hints

- Search for key prefixes, "Bearer " and VITE_ names ending in KEY or SECRET.
- Build fake keys at runtime ("sk-" + "demo" * 6) so no key-like text is saved in your files.

Problem 6: SmartDesk Tickets API

Level: Challenging | **Concepts:** FastAPI CRUD, CORS

Build GET, POST and PATCH /api/tickets in FastAPI with Pydantic models (status open, pending or closed; priority low, medium or high) and CORS for http://localhost:5173. Test with TestClient.

Sample run / expected result

```
POST /api/tickets {"title": "No signal"} -> 201
PATCH /api/tickets/1 {"status": "closed"} -> 200
PATCH /api/tickets/1 {"status": "done"} -> 422
```

Hints

- Use Literal["open", "pending", "closed"] for the status field.

Problem 7: Connect React to FastAPI

Level: Stretch | **Concepts:** fetch GET, POST, PATCH; optimistic UI

Load tickets from the API on mount, create new tickets with POST, change priority and close tickets with PATCH. Make Close optimistic: update the screen first, then roll back with an error message if the request fails.

Sample run / expected result

```
Close -> ticket greyed instantly -> API fails -> ticket restored + "Could not save"
```

Hints

- Keep the previous list in a variable before changing state.